MAINCRAFTS TECHNOLOGY

Automatic Light Detection System using ESP32 & LDR

Embedded Systems – Task 2

Project

Automatic Light Detection System using ESP32 and LDR Sensor

Submitted by

Shree Harshan K

Embedded System Developer (Trainee)

Date: 07-07-2026

Table of Contents

1. Introduction to Embedded Systems

An embedded system is a combination of hardware and software designed to carry out a specific task within a larger device or process. Unlike general-purpose computers, embedded systems are purpose-built, meaning their hardware and code are tailored to a single job rather than running a variety of unrelated applications.

This task builds on the embedded systems concepts covered in Task 1, focusing this time on analog sensor interfacing rather than digital sensing. The same core principles apply: a microcontroller reads input from a sensor, makes a decision based on that input, and drives an output device accordingly.

1.1 Why Analog Sensing Matters

Many real-world quantities, such as light intensity, sound level, or pressure, do not naturally exist as simple ON/OFF signals. Analog sensors capture this continuous range of values and convert it into a voltage that a microcontroller's Analog-to-Digital Converter (ADC) can read. This project explores exactly that kind of interfacing using an LDR (Light Dependent Resistor) and the ESP32's built-in ADC.

1.2 Relevance to IoT and Automation

Threshold-based decision making, where a system compares a live sensor reading against a fixed reference value, is one of the most common patterns in embedded and IoT design. It forms the basis of automatic lighting systems, smart irrigation, climate control, and countless other automation applications, making it an important concept to understand early on.

2. Component Study

2.1 ESP32 DevKit V1

The ESP32 DevKit V1 is a development board built around the ESP32 microcontroller, featuring a dual-core processor, built-in Wi-Fi and Bluetooth, and multiple ADC-capable GPIO pins. For this project, its analog input capability on GPIO34 was the key feature used, allowing direct reading of the LDR's analog output without any additional conversion circuitry.

2.2 LDR Sensor Module (Analog Output)

A Light Dependent Resistor (LDR) is a photoresistor whose resistance decreases as light intensity increases. The sensor module used in this project outputs this changing resistance as an analog voltage, which the ESP32 reads as a value between 0 and 4095 (12-bit ADC resolution). A bright environment produces a high reading, while a dark environment produces a low reading. The module also includes a digital output (DO) pin, but since this project required fine-grained light-level readings rather than a simple ON/OFF trigger, only the analog output (AO) pin was used.

2.3 LED (Output Indicator)

The LED serves as the system's output device, visually representing the ESP32's decision. It lights up when the surroundings are detected as dark and switches off automatically once sufficient ambient light is detected.

2.4 220Ω Resistor

The 220Ω resistor is placed in series with the LED to limit the current flowing through it. Without this resistor, the LED could draw excessive current from the GPIO pin and potentially damage either the LED or the microcontroller.

2.5 Breadboard and Jumper Wires

The breadboard was used to assemble the circuit without soldering, while jumper wires provided the electrical connections between the ESP32, the LDR module, and the LED-resistor combination. This setup made it easy to test and adjust the circuit during simulation.

3. Project Details

3.1 Project Title

Automatic Light Detection System using ESP32 and LDR Sensor

3.2 Objective

To design and simulate an automatic light detection system using an ESP32 microcontroller and an LDR sensor. The system continuously monitors ambient light intensity and automatically controls an LED based on the detected light level, demonstrating analog sensor interfacing, threshold-based decision making, and GPIO output control.

3.3 Components Used

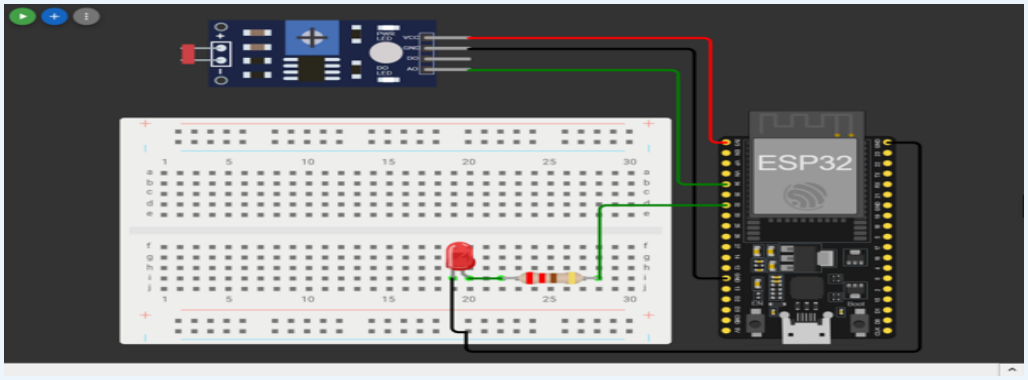
- ESP32 DevKit V1 – central microcontroller
- LDR Sensor Module (Analog Output)
- LED – output indicator
- 220Ω Resistor – current limiting
- Breadboard
- Jumper Wires

3.4 Software and Tools Used

- Wokwi Online Simulator – circuit design and simulation
- Arduino IDE (Arduino Framework) – code development
- GitHub – version control and project documentation

3.5 Circuit Diagram

The circuit was designed and tested in the Wokwi ESP32 online simulator. The diagram below shows the LDR module and LED-resistor connections to the ESP32.



3.6 Pin Connections

Component	ESP32 Pin
LDR Module – VCC	3.3V
LDR Module – GND	GND
LDR Module – AO	GPIO 34
LDR Module – DO	Not connected
LED – Anode (+)	GPIO 32 (via 220Ω resistor)
LED – Cathode (–)	GND

Note: the LDR module's digital output (DO) pin was intentionally left unconnected since only analog readings were required for this project's threshold-based logic.

4. Working Principle

The LDR sensor detects the surrounding light intensity and produces an analog voltage proportional to the amount of light falling on it. The ESP32 continuously reads this analog voltage using the analogRead() function on GPIO34, which returns a value between 0 and 4095 based on the 12-bit ADC.

A threshold value of 2000 was selected after observing the sensor's behaviour during simulation testing. This value sits comfortably between the readings observed under bright and dark conditions, giving the system a reliable margin for decision-making.

LDR Reading	System Decision
Below 2000	Environment considered DARK – LED turned ON
Above 2000	Environment considered BRIGHT – LED turned OFF

The current sensor value, the inferred environment status, and the LED state are printed to the Serial Monitor once every second, making it easy to observe the system's behaviour in real time during testing.

4.1 Program Logic

- Initialize Serial communication
- Configure the LED pin as an output
- Continuously read the analog value from the LDR sensor
- Compare the sensor value with the predefined threshold
- Turn the LED ON when the surroundings are dark
- Turn the LED OFF when the surroundings are bright
- Display the sensor reading and system status on the Serial Monitor
- Repeat the process every second

5. Source Code with Explanation

The following Arduino/C++ code was written for the ESP32. Each section is commented to explain its purpose.

```
// ── Pin Definitions ──────────────────────────────────────────────
#define LDR_PIN  34   // LDR module analog output (AO) connected to GPIO34
#define LED_PIN  32   // LED connected to GPIO32 via 220-ohm resistor

// ── Threshold Value ──────────────────────────────────────────────
#define LIGHT_THRESHOLD 2000  // Below this = dark, above this = bright

// ── setup() - runs once on power-up ──────────────────────────────
void setup() {
  Serial.begin(115200);     // Start serial monitor at 115200 baud

  pinMode(LED_PIN, OUTPUT); // Configure LED pin as output
  digitalWrite(LED_PIN, LOW); // Ensure LED is off at startup

  // Note: analogRead() pins do not need pinMode() on ESP32
  Serial.println("Automatic Light Detection System Initialized");
}

// ── loop() - runs repeatedly ─────────────────────────────────────
void loop() {
  // Read the analog value from the LDR (range: 0-4095)
  int ldrValue = analogRead(LDR_PIN);

  // Compare against threshold and decide LED state
  if (ldrValue < LIGHT_THRESHOLD) {
    // ── DARK CONDITION ──────────────────────────────────────────
    digitalWrite(LED_PIN, HIGH);   // Turn LED ON

    Serial.print("LDR Value: ");
    Serial.print(ldrValue);
    Serial.println(" | Environment: DARK | LED: ON");
  } else {
    // ── BRIGHT CONDITION ────────────────────────────────────────
    digitalWrite(LED_PIN, LOW);    // Turn LED OFF

    Serial.print("LDR Value: ");
    Serial.print(ldrValue);
    Serial.println(" | Environment: BRIGHT | LED: OFF");
  }

  delay(1000);  // Wait 1 second before the next reading
}
```

5.1 Code Walkthrough

The code begins by defining the two GPIO pins used in the project: GPIO34 for the LDR's analog output and GPIO32 for the LED. The threshold value of 2000 is defined as a constant so it can be tuned easily if the lighting conditions change.

In setup(), the LED pin is configured as an output and explicitly set LOW so it does not light up unexpectedly when the board powers on. Unlike digital pins, ESP32 analog input pins do not require a pinMode() call before use.

Inside loop(), analogRead(LDR_PIN) returns the current light reading. This value is compared against LIGHT_THRESHOLD: if it falls below the threshold, the environment is treated as dark and the LED is switched on; otherwise, the environment is treated as bright and the LED is switched off. In both cases, the reading and the resulting decision are printed to the Serial Monitor before the loop pauses for one second and repeats.

6. Results

The system was tested in Wokwi by simulating both bright and dark lighting conditions and observing the Serial Monitor output.

6.1 Bright Condition

- LDR Value: approximately 3017
- Environment: BRIGHT
- LED State: OFF

6.2 Dark Condition

- LDR Value: approximately 32
- Environment: DARK
- LED State: ON

6.3 Sample Serial Monitor Output

Condition	Serial Monitor Output
Bright (LDR ≈ 3017)	LDR Value: 3017 | Environment: BRIGHT | LED: OFF
Dark (LDR ≈ 32)	LDR Value: 32 | Environment: DARK | LED: ON

The LED responded correctly and immediately in both directions, confirming that the threshold value of 2000 was well chosen for distinguishing between bright and dark conditions in the simulated environment.

7. Features

- Automatic light detection based on ambient brightness
- Analog sensor interfacing with the ESP32's built-in ADC
- Real-time sensor monitoring via the Serial Monitor
- Automatic LED control based on a tunable threshold value
- Serial output for debugging and observation during testing
- Entirely simulated and verified using the Wokwi platform

8. Real-World Applications

Light-sensing automation like this has a number of practical, energy-saving applications:

Automatic Street Lighting

Streetlights can switch on automatically at dusk and off at dawn without manual intervention or fixed timers, adapting naturally to seasonal daylight changes.

Smart Home Lighting

Indoor lights can respond to natural light levels, reducing unnecessary electricity use during daylight hours while ensuring rooms are lit when needed.

Corridor and Staircase Lighting

Shared spaces in buildings often only need lighting during low-light periods. Automatic detection avoids lights being left on unnecessarily throughout the day.

Energy-Saving Lighting Systems

By only activating lighting when genuinely required, systems built on this principle directly reduce energy consumption in homes, offices, and public infrastructure.

Light-Sensitive Automation Projects

The same threshold-based approach can be extended to greenhouse shading systems, automatic curtain controllers, and photography equipment that needs to respond to ambient light.

9. Learning Outcomes & Conclusion

Through this project, I learned how to interface an analog sensor with the ESP32, read analog input values using analogRead(), and apply threshold-based decision making to control a GPIO output. Selecting an appropriate threshold value also gave me a practical understanding of sensor calibration, since the right value had to be found by observing actual readings rather than guessing one in advance.

Working through the circuit wiring, particularly deciding to use the LDR module's analog output instead of its digital output, helped me understand the trade-off between simple ON/OFF sensing and more flexible analog sensing. The current-limiting resistor on the LED was also a good practical reminder of basic circuit protection.

The final simulated system met all the project requirements: it continuously monitored ambient light, made a clear ON/OFF decision for the LED based on a well-chosen threshold, and reported its status over Serial for easy verification. This project improved my understanding of embedded programming, sensor calibration, and hardware-software integration, and builds directly on the foundational concepts covered in Task 1.

Submitted by: Shree Harshan K

Task: Embedded Systems Task 2 – Maincrafts Technology

Date: 07-07-2026